

Intent Expansion Pipeline: Approach and Learning

Table of Contents

1. [Executive Summary](#)
 2. [Problem Understanding](#)
 3. [Approach & Architecture](#)
 4. [Workflow Design](#)
 5. [Key Design Decisions](#)
 6. [Guardrails & Fallbacks](#)
 7. [Assessment Metrics](#)
 8. [Limitations & Edge Cases](#)
 9. [Scalability Considerations](#)
 10. [Future Improvements](#)
-

Executive Summary

This pipeline addresses the critical challenge of **intent discovery and refinement** in conversational AI systems. The solution provides a **scalable, structured workflow** that:

- Analyzes customer messages in batches (not bulk processing)
- Identifies patterns suggesting new or split-worthy intents
- Uses LLMs intelligently with structured prompts (not dumping all data)
- Applies multiple guardrails to prevent over-clustering
- Provides quantitative and qualitative justification for proposals
- Evaluates proposals using custom assessment metrics

Key Innovation: The pipeline balances automation (LLM-powered analysis) with deterministic guardrails to ensure proposals are meaningful, distinct, and won't cause classification failures in smaller LLM models.

Problem Understanding

Core Challenge

- **Current State:** Broad intents limit personalization and automation capabilities
- **Goal:** Discover missing intents and refine existing ones

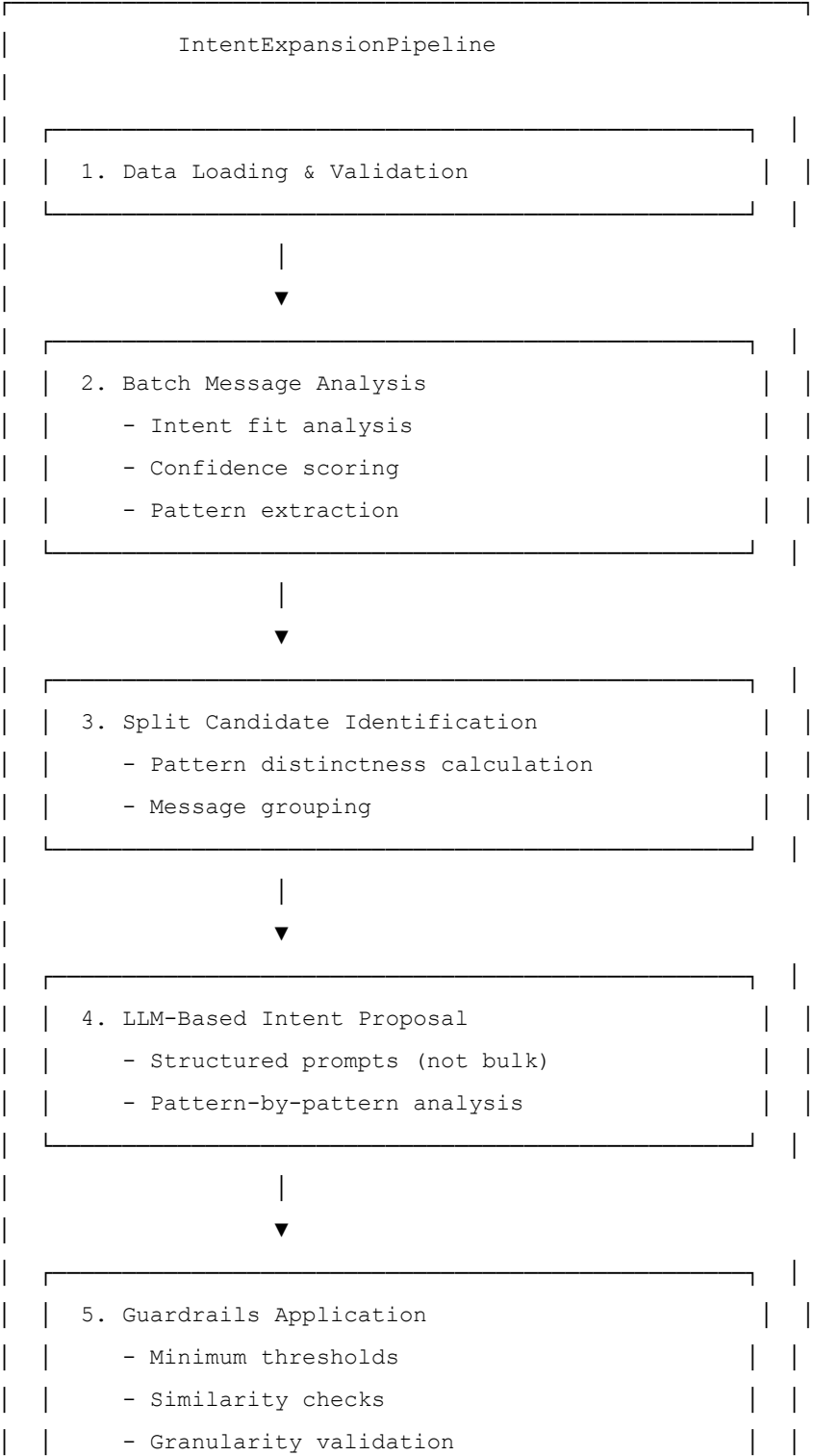
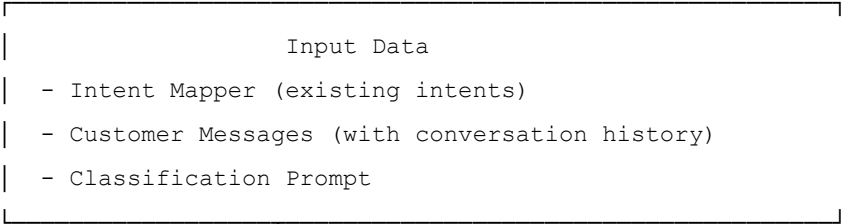
- **Constraint:** Too many granular intents can cause smaller LLMs (gpt-4o-mini, 5.1-nano) to fail
- **Requirement:** Each token passed to the model must be "weighed in gold"

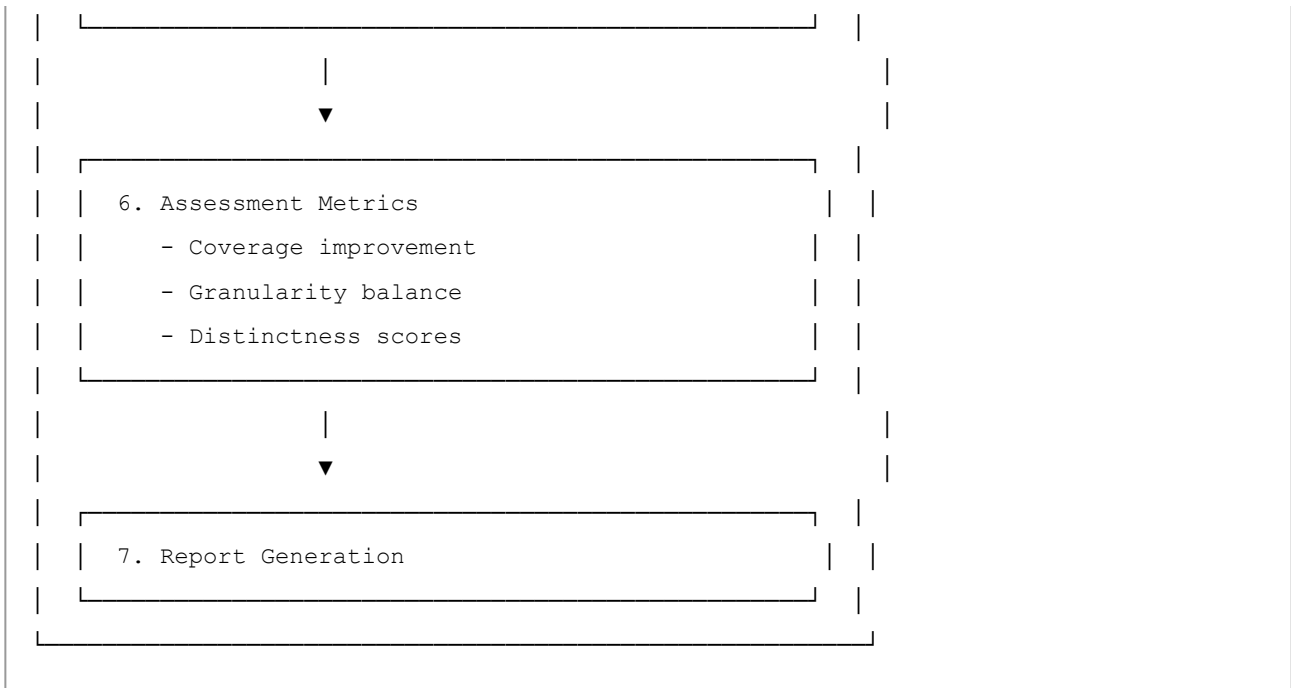
Key Insights

1. **Intent Granularity Balance:** Need specificity without over-clustering
 - Example: Don't split "ingredients", "components", "materials" separately
 - But do split "Product Usage" from "Product Info" if patterns are distinct
 2. **Scalability Requirement:** Must handle hundreds → thousands of messages
 - Cannot pass all data to LLM at once
 - Need batch processing and pattern aggregation
 3. **Evidence-Based Proposals:** Each proposal needs justification
 - Quantitative: Message counts, confidence scores, distinctness metrics
 - Qualitative: Sample messages, pattern descriptions
-

Approach & Architecture

High-Level Architecture





Component Breakdown

1. IntentAnalyzer

- **Purpose:** Core pattern analysis without LLM dependency
- **Key Functions:**
 - Key phrase extraction
 - Intent fit analysis (confidence scoring)
 - Pattern distinctness calculation
- **Why:** Provides fast, deterministic analysis before expensive LLM calls

2. LLMIntentProposer

- **Purpose:** Intelligent intent proposal using LLMs
- **Key Design:**
 - **Structured prompts:** Not bulk processing
 - **Pattern-by-pattern:** Analyze one pattern at a time
 - **Token-efficient:** Summarize intents, limit sample messages
- **Why:** Leverages LLM intelligence while maintaining scalability

3. IntentExpansionPipeline

- **Purpose:** Orchestrates the entire workflow
 - **Key Features:**
 - Batch processing for scalability
 - Guardrails for quality control
 - Assessment metrics integration
-

Workflow Design

Step-by-Step Workflow

Phase 1: Data Loading & Preparation

1. Load intent mapper (existing intents)
2. Load customer messages
3. Validate data structure
4. Initialize analyzers with configuration

Phase 2: Message Analysis (Batch Processing)

- For each batch of messages (default: 50):
1. Extract message text and current intent classification
 2. Analyze intent fit using keyword matching and pattern detection
 3. Calculate confidence score
 4. Group messages by current intent
 5. Track low-confidence messages

Scalability: Processes in batches, not all at once. Memory-efficient.

Phase 3: Pattern Identification

- For each intent category:
1. Extract key phrases from messages
 2. Group messages by dominant phrases
 3. Calculate pattern distinctness
 4. Identify patterns with sufficient message count (threshold: 5)

Rationale: Patterns with few messages might be noise. Threshold prevents over-clustering.

Phase 4: LLM-Based Proposal Generation

For each significant pattern:

1. Build structured prompt with:
 - Summary of existing intents (not full list)
 - Sample messages (limited to 10, truncated to 200 chars)
 - Clear constraints (avoid over-granularity)
2. Call LLM with structured output format
3. Parse response into IntentProposal object
4. Add quantitative evidence (message count, distinctness)

Key Design Decision:

- **NOT** passing all messages to LLM
- **NOT** passing full intent definitions
- **YES** summarizing and limiting context
- **YES** structured output format

Phase 5: Guardrails Application

For each proposal:

1. Check minimum message threshold
2. Check minimum confidence score
3. Check for similar existing intents (similarity > 0.7)
4. Check for over-granularity (too many specific terms)
5. Filter out proposals that fail any guardrail

Why Guardrails Matter: Prevents:

- Proposing intents for rare edge cases
- Creating duplicate intents
- Over-clustering (ingredients/components/materials split)

Phase 6: Assessment & Reporting

1. Calculate coverage improvement metrics
2. Calculate granularity balance score
3. Calculate distinctness scores
4. Generate recommendations
5. Create comprehensive JSON report

Key Design Decisions

1. Batch Processing Over Bulk Processing

Decision: Process messages in batches of 50 (configurable)

Rationale:

- Scalable to thousands of messages
- Memory-efficient
- Allows progress tracking
- Can be parallelized in future

Trade-off: Slight overhead from batch management, but essential for scalability.

2. Two-Stage Analysis (Deterministic + LLM)

Decision: Use deterministic analysis first, then LLM for proposals

Rationale:

- Fast pattern detection without LLM costs
- LLM only called for significant patterns
- Reduces token usage
- More deterministic and debuggable

Trade-off: Might miss subtle patterns, but guardrails catch edge cases.

3. Structured LLM Prompts (Not Bulk Data)

Decision: Summarize intents, limit sample messages, use structured output

Rationale:

- Token efficiency (critical for cost and model limits)
- Better LLM focus (not overwhelmed by data)
- Structured output easier to parse
- Aligns with "each word weighed in gold" requirement

Example:

- ☐ BAD: Pass all 200 messages + full intent definitions
- ☐ GOOD: Pass summary of intents + 10 sample messages (truncated)

4. Multiple Guardrails

Decision: Apply 4+ guardrails before accepting proposals

Rationale:

- Prevents over-clustering (critical constraint)
- Ensures quality (minimum thresholds)
- Avoids duplicates (similarity checks)
- Maintains balance (granularity validation)

5. Assessment Metrics

Decision: Build custom metrics to evaluate proposals

Rationale:

- Quantifies improvement (coverage)
 - Measures balance (granularity)
 - Validates distinctness
 - Provides actionable recommendations
-

Guardrails & Fallbacks

Guardrail 1: Minimum Message Threshold

- **Rule:** Propose intent only if ≥ 5 messages show the pattern
- **Rationale:** Prevents proposing intents for rare edge cases
- **Configurable:** `min_message_threshold` parameter

Guardrail 2: Minimum Confidence Score

- **Rule:** Propose intent only if confidence ≥ 0.7
- **Rationale:** Ensures proposals are well-justified
- **Configurable:** `min_confidence` parameter

Guardrail 3: Similarity Check

- **Rule:** Reject if similarity with existing intent > 0.7
- **Rationale:** Prevents duplicate intents
- **Method:** Keyword overlap calculation

Guardrail 4: Granularity Validation

- **Rule:** Reject if too many specific terms (ingredients/components/materials)
- **Rationale:** Prevents over-clustering that causes LLM failures
- **Method:** Count specific terms in description

Guardrail 5: Description Quality

- **Rule:** Reject if description too short (< 5 words)
- **Rationale:** Ensures meaningful intent definitions
- **Method:** Word count check

Fallback Strategy

1. **LLM Failure:** If LLM call fails, skip that pattern (log error, continue)
 2. **No Patterns Found:** Return empty proposals with explanation
 3. **All Proposals Rejected:** Report why (all guardrails failed)
 4. **Low Confidence:** Include in report but mark as "needs review"
-

Assessment Metrics

1. Coverage Improvement

- **Metric:** % of messages that can be confidently classified
- **Calculation:** (High confidence messages + messages for new intents) / Total
- **Goal:** Increase coverage without sacrificing accuracy

2. Granularity Balance Score

- **Metric:** Balance between specificity and over-clustering
- **Calculation:** Penalty for too many intents, reward for meaningful splits
- **Goal:** Keep balance score > 0.7

3. Distinctness Scores

- **Metric:** How distinct each proposal is from existing intents
- **Calculation:** $1 - (\text{keyword overlap with existing intents})$
- **Goal:** Overall distinctness > 0.6

4. Confidence Distribution

- **Metric:** Distribution of confidence scores
- **Calculation:** Mean, min, max, and distribution buckets
- **Goal:** Mean confidence > 0.7, high confidence count > 50%

Overall Quality Score

Weighted combination of all metrics:

- Coverage improvement: 30%
- Granularity balance: 30%

- Distinctness: 30%
 - Confidence: 20%
-

Limitations & Edge Cases

Limitations

1. Keyword-Based Analysis

- **Issue:** Simple keyword matching might miss semantic similarity
- **Impact:** Might propose intents that are semantically similar
- **Mitigation:** LLM analysis catches semantic patterns, similarity check filters duplicates

2. Pattern Extraction Heuristics

- **Issue:** Key phrase extraction is rule-based, not ML-based
- **Impact:** Might miss subtle patterns
- **Mitigation:** LLM analysis provides semantic understanding

3. LLM Dependency

- **Issue:** Requires LLM API access and costs
- **Impact:** Cannot run without API key
- **Mitigation:** Mock mode for testing, clear error messages

4. Language Assumptions

- **Issue:** Assumes English language
- **Impact:** Won't work for other languages
- **Mitigation:** Can be extended with language detection

Edge Cases Handled

1. **Empty Messages:** Skipped with warning
 2. **Missing Intent Classification:** Handled gracefully (low confidence)
 3. **Very Short Messages:** Included but might have lower confidence
 4. **Ambiguous Messages:** Low confidence, grouped for review
 5. **LLM Timeout/Error:** Logged, pattern skipped, pipeline continues
 6. **No Patterns Found:** Returns empty proposals with explanation
 7. **All Proposals Rejected:** Detailed report on why
-

Scalability Considerations

Current Scalability

- **Tested:** 200 messages (assignment requirement)
- **Designed for:** Thousands of messages
- **Batch size:** 50 messages (configurable)
- **Memory:** $O(\text{batch_size})$, not $O(\text{total_messages})$

Scalability Features

1. **Batch Processing:** Processes in chunks, not all at once
2. **Pattern Aggregation:** Groups messages before LLM calls
3. **Limited LLM Context:** Summarizes intents, limits samples
4. **Efficient Data Structures:** Uses defaultdict, Counter for fast lookups

Future Scalability Improvements

1. **Parallel Processing:** Process batches in parallel
 2. **Caching:** Cache LLM responses for similar patterns
 3. **Incremental Processing:** Process new messages incrementally
 4. **Database Integration:** Store results in database for large datasets
-

Future Improvements

Short-Term

1. **Embedding-Based Similarity:** Replace keyword matching with embeddings
2. **Multi-Language Support:** Add language detection and processing
3. **Interactive Review:** Allow human review before finalizing proposals
4. **A/B Testing:** Test new intents on subset before full deployment

Medium-Term

1. **Active Learning:** Learn from human feedback on proposals
2. **Temporal Analysis:** Track intent evolution over time
3. **Domain Adaptation:** Adapt to different business domains
4. **Confidence Calibration:** Better confidence score calibration

Long-Term

1. **Automated Deployment:** Auto-deploy validated intents
2. **Continuous Monitoring:** Monitor intent performance post-deployment
3. **Intent Lifecycle Management:** Handle intent deprecation, merging
4. **Multi-Model Support:** Support different LLM providers seamlessly

Conclusion

This pipeline provides a **structured, scalable approach** to intent expansion that:

- ❑ **Balances automation with control** (LLM + guardrails)
- ❑ **Scales to thousands of messages** (batch processing)
 - ❑ **Prevents over-clustering** (multiple guardrails)
- ❑ **Provides evidence-based proposals** (quantitative + qualitative)
 - ❑ **Evaluates its own output** (assessment metrics)

The design prioritizes **workflow thinking** and **attention to detail** over perfect accuracy, recognizing that intent expansion is an iterative process that benefits from human review and refinement.

Usage Example

```
from intent_expansion_pipeline import IntentExpansionPipeline

# Initialize pipeline
config = {
    'min_message_threshold': 5,
    'min_confidence': 0.7,
    'openai_api_key': 'your-key-here',
    'llm_model': 'gpt-4o-mini'
}

pipeline = IntentExpansionPipeline(config)

# Run pipeline
proposals, report = pipeline.run('inputs_for_assignment.json', batch_size=50)

# Review proposals
for proposal in proposals:
    print(f"{proposal.name} ({proposal.id}): {proposal.description}")
    print(f"  Confidence: {proposal.confidence_score:.2f}")
    print(f"  Messages: {proposal.quantitative_evidence['message_count']}")
    print(f"  Rationale: {proposal.rationale}")
```

Author: AI Workflow Analyst Intern Candidate

Date: 2024

Version: 1.0

